

정혜원

PROGRAMMER

물리 기반 액션 구현부터,
여러 시스템이 얹힌 게임의 상태 관리까지 —
직접 구현하고, 문제가 생기면 끝까지 추적해 해결해왔습니다.

Unity

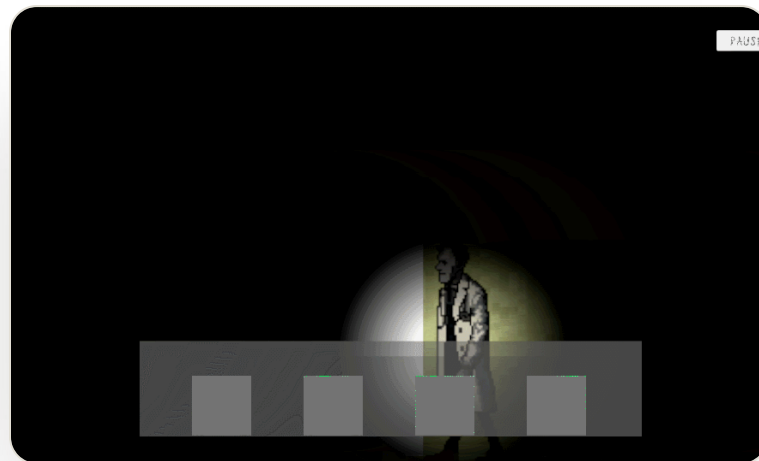
C#

Team Project

Debugging

GITHUB
github.com/hyewang5617

GAME PORTFOLIO
hyewang5617.github.io/Portfolio



중학생 때 만든 첫 게임 이후, 지금까지 꾸준히 게임을 만들어왔습니다

Unity와 C#을 중심으로 GameMaker Studio · p5.js · Python 등 여러 환경을 넘나들며 게임을 만들어 왔습니다. 플랫폼어, RPG, 로그라이크, 퍼즐, 공포 게임까지 장르를 가리지 않고 아이디어를 직접 구현해보는 과정을 즐겼고, 혼자 만든 개인 프로젝트부터 여러 명이 함께한 팀 프로젝트까지 다양한 형태로 개발 경험을 쌓아왔습니다.

TECH STACK

Unity

C#

GameMaker Studio

p5.js

Python

Processing

OpenCV

Git / GitHub

GAME MAKING TIMELINE — 2020 → 2026

2020

포도씨의 모험 (3D)

Unity로 만든 첫 번째 게임 · 3D 플랫폼어 · 캐릭터·디자인 직접 제작

2020

포도씨의 모험 (2D)

Unity · 더블점프 하드코어 플랫폼어 · 도트 직접 제작

2020

테트리스

Unity · 학교 온라인 소프트웨어 강의 프로젝트

2021

뽕머랭님 팬게임

Unity · 탑다운 RPG · 직접 기획한 팬게임

2021

Shark Timer Game

p5.js · 공동 제작 브라우저 게임

2022

빛이있으라

GameMaker Studio · 팀장 · 4인 팀 로그라이크

2023

Arrow Master

p5.js · 2인용 궁술 대결 게임

2023

더 인기, 더 팔로워

Python · tkinter 퀴즈 게임

2024

내 과일을 받아줘

Processing · 외주 의뢰 캐주얼 게임

2024

삐에로의 풍선

Unity · 2D 횡스크롤 공포 게임

2024

뱀서라이크

Unity · Vampire Survivors 류 로그라이크 슈터

2025

함정 피하기 게임

Unity · 2D 플랫폼어

2025

Crazy Commuter

Unity 3D · 2인 팀 · 러너·플랫폼어 → P3

2026

Cooking Papa

Python · OpenCV · 웹캠 손동작 인식 게임 → P7

2026

PH-1

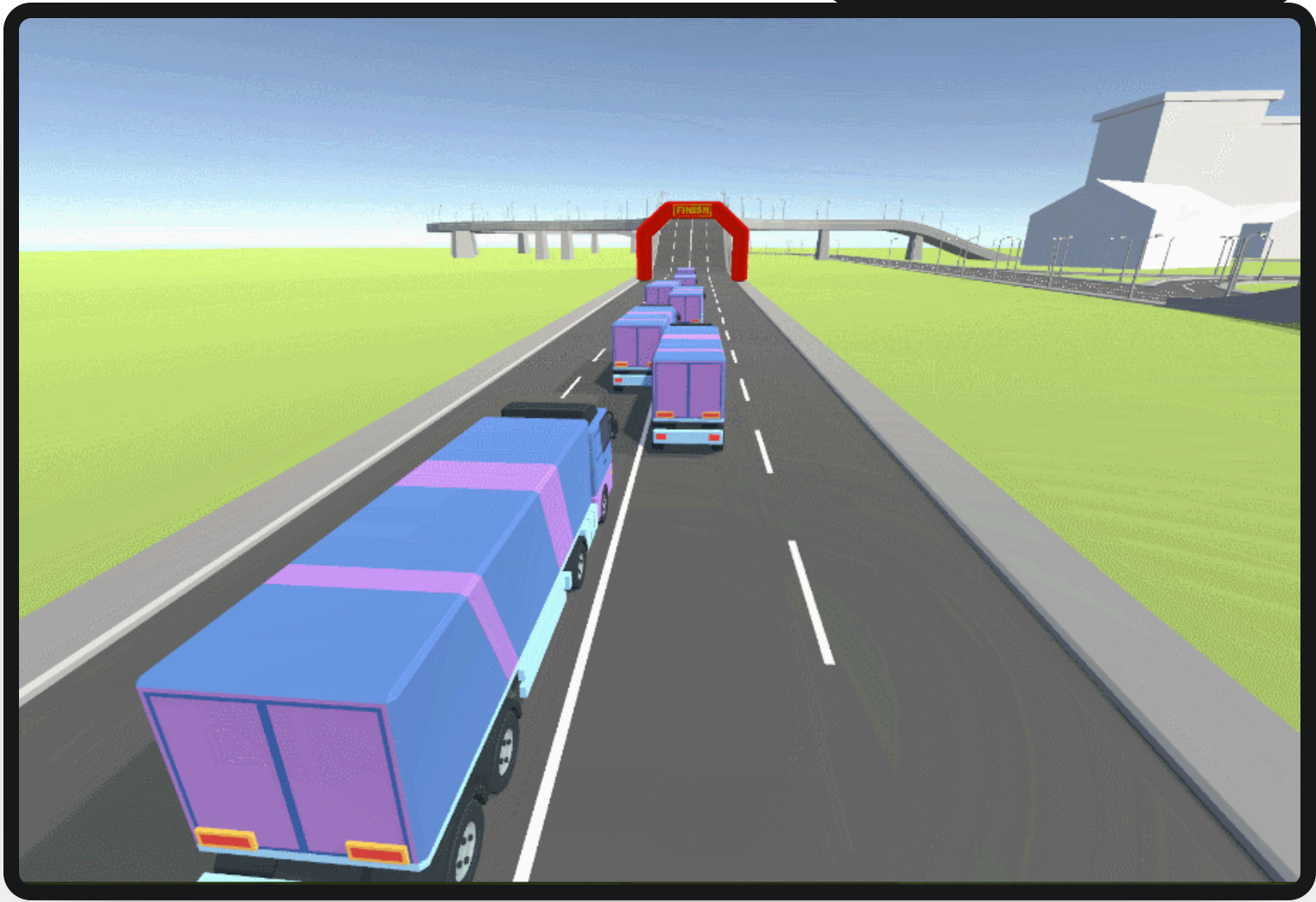
Unity 2D · 4인 팀 · 퍼즐 어드벤처 (개발중) → P5

움직이는 차량을 장애물이 아닌 '발판'으로

게임프로그래밍 수업에서 제시된 러너 게임이라는 주제에서 출발해,
ClusterTruck의 플레이 방식에 착안한 3D 러너·플랫폼 게임 기획했습니다.
플레이어는 도로를 달리는 차량 위를 뛰어다니며 결승점까지 이동합니다.

단순히 정해진 발판을 따라가는 방식이 아니라, 차량의 생성 위치와
이동 경로에 무작위성을 적용해 같은 스테이지에서도
매번 다른 상황이 발생하도록 설계했습니다.

2인 팀으로 개발했으며, 게임 전체 콘셉트 기획과 함께
플레이어 조작, 차량 이동·생성 시스템, Stage 1·2, 인게임 UI 전반을 담당했습니다.



PLAYER

기본 이동·점프·더블점프 등 차량 사이를 이동하기 위한
플레이어 조작 구현

VEHICLE

Rigidbody 기반 차량 이동과 웨이포인트 경로 주행 구
현

RANDOM

차량 생성 위치와 이동 경로에 무작위성을 적용해 반복
플레이 변화

LEVEL DESIGN

Stage 1 — 직선 도로 기본 조작 학습 · Stage 2 — 급
커브·차량 충돌 활용

MY ROLE

Game Concept / Planning

Player Movement

Vehicle System

Vehicle Spawner

Stage 1·2 Level Design

In-game UI

TECH

Unity

C#

Git/GitHub

Rigidbody

Physics.OverlapSphere

Quaternion.Slerp

PROBLEM & SOLUTION

차량을 움직이는 것보다 '자연스럽게 움직이게 하는 것'이 더 어려웠습니다

01

랜덤 생성했는데 결국 한 줄로 모인 차량

서로 다른 위치에서 출발해도 모든 차량이 동일한 웨이포인트를 향하면서 결국 일렬로 수렴했습니다.

SOLUTION

웨이포인트 주변에 차량별 랜덤 오프셋을 적용해, 차량마다 조금씩 다른 경로로 주행하도록 수정했습니다.

02

90° 커브에서 갑자기 꺾이며 충돌하는 차량

웨이포인트에 도착한 뒤 다음 방향으로 회전하도록 하니, 급커브에서 방향을 크게 틀며 뒤따르던 차량과 충돌했습니다.

SOLUTION

도착 전부터 다음 진행 방향을 미리 반영하고, 회전을 한 번에 바꾸지 않고 점진적으로 보간하도록 수정했습니다.

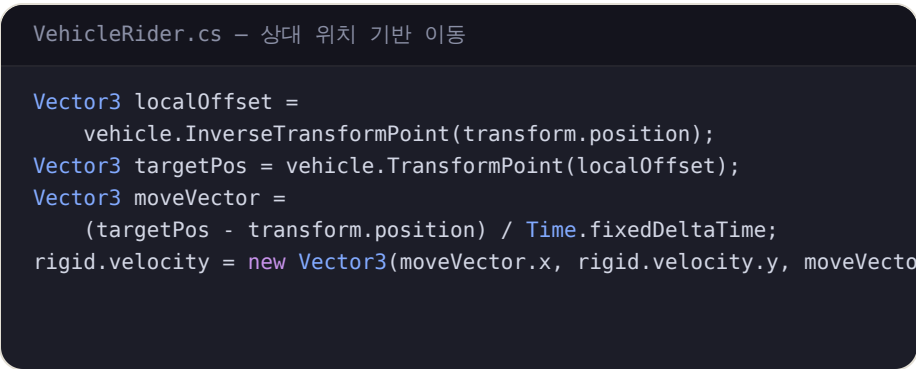
03

차량 위 플레이어의 Rigidbody가 불안정했던 문제

`transform.SetParent(vehicle.transform)` 로 차량의 자식으로 만들었더니, Rigidbody 물리 충돌이 불안정해졌습니다.

SOLUTION

부모-자식 관계 대신 차량 기준 상대 위치를 저장하고, 이동량을 속도로 환산해 `Rigidbody.velocity`에 적용했습니다.



PHYSICS TROUBLESHOOTING — 물리 기반 게임에서 추가로 처리한 예외 상황

Fast Object Collision

빠르게 움직이는 차량에서 충돌 판정이 누락되는 문제 해결

Spawn Collision

`Physics.OverlapSphere` 로 생성 위치를 검사해 차량이 겹쳐 생성되는 것을 방지

Traffic Jam

차량이 충돌 후 장시간 정체되면 폭발 처리해 진행 불가 상태 방지

4 STAGES COMPLETED

2인 팀으로 4개 스테이지 모두 플레이 가능한 상태까지 완성

2nd PLACE

수강생 상호 플레이 평가 2위



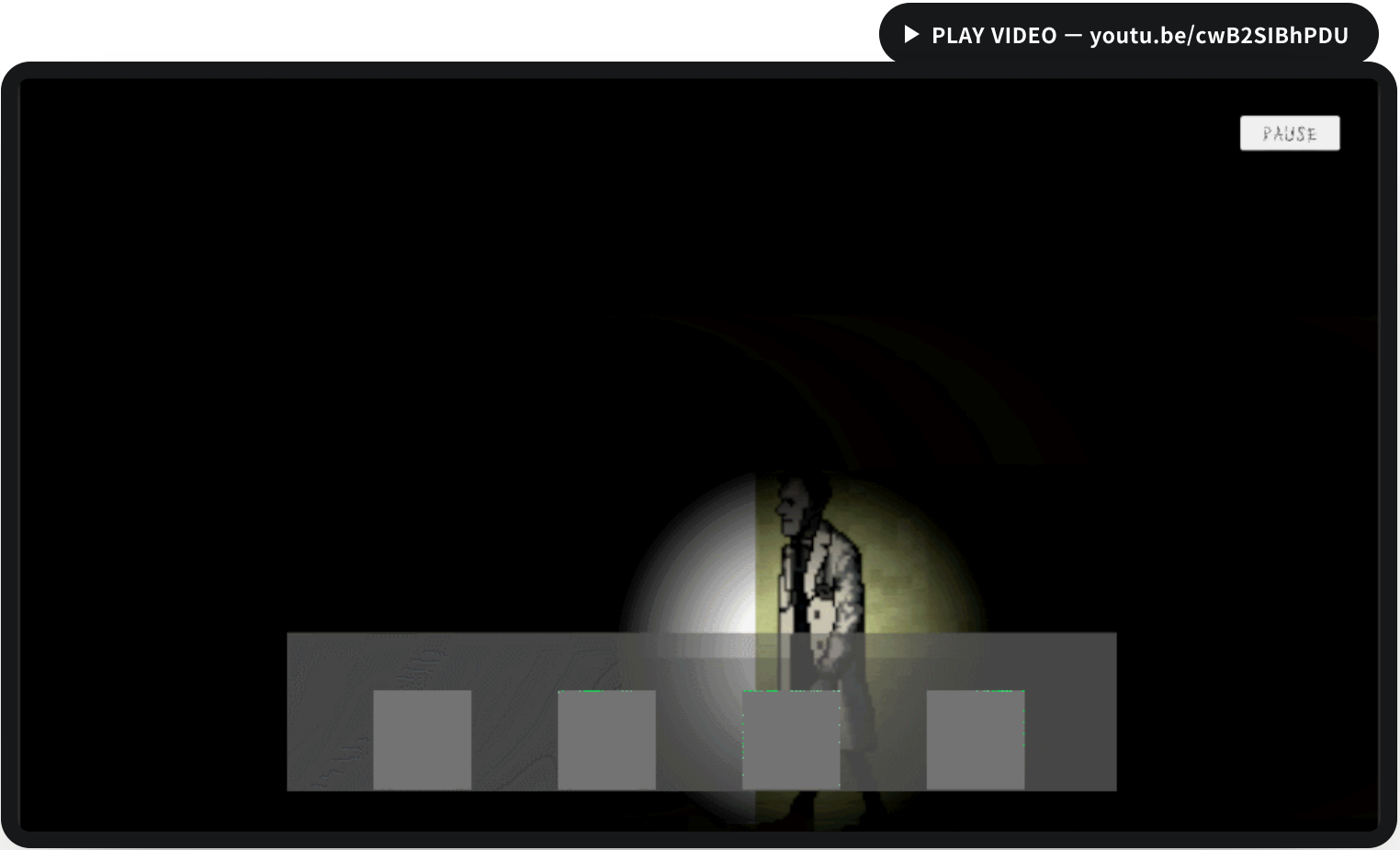
WHAT I LEARNED — 기능이 단순히 작동하는 것과 실제 플레이 환경에서 안정적으로 동작하는 것은 다르다는 점을 배웠습니다. 이후에는 기능을 구현한 뒤 직접 반복 플레이하며 예외 상황을 찾고, 원인을 물리·이동 로직의 흐름에서 추적해 수정하는 방식으로 개발했습니다.

퍼즐 하나를 만드는 것에서 시작해 게임의 '상태'를 관리하는 것까지

Unity와 C#으로 제작 중인 2D 퍼즐 어드벤처 게임입니다.
플레이어가 공간을 탐색하며 아이템을 획득하고,
여러 퍼즐과 이벤트를 해결하며 이야기를 진행하는 구조입니다.

프로젝트에서 인벤토리와 아이템 시스템, 리듬 퍼즐, 몬스터 퍼즐,
세이브·로드 및 게임 진행 상태 관리를 중심으로 구현했습니다.

게임의 규모가 커지면서 단순히 하나의 기능을 구현하는 것보다,
서로 다른 시스템의 상태가 썸을 넘어 자연스럽게 이어지도록
만드는 과정을 경험했습니다.



아이템 획득



세이브 · 로드

INVENTORY

아이템 획득·보관·사용이 가능한 인벤토리 시스템 구현

PUZZLE

접근 링을 이용한 리듬 퍼즐과 몬스터 추적 퍼즐 구현

SAVE & LOAD

플레이어 상태와 인벤토리, 게임 진행 정보를 저장·복원

GAME STATE

ScriptableObject 기반 GameFlag로 이벤트 완료 상태 관리

MY ROLE

Inventory & Item System

Rhythm Puzzle

Monster Puzzle

Save & Load

GameFlag / Event State

Spawn & Scene Transition Debugging

TECH

Unity

C#

Git/GitHub

ScriptableObject

Cinemachine

여러 시스템이 연결되자 예상하지 못한 문제가 생겼습니다

PROBLEM & SOLUTION

01

이벤트를 완료했는데 다시 돌아오면 처음부터 실행됐습니다

씬이 늘어나면서 특정 이벤트를 이미 완료했는지에 따라 대화·시네마틱·씬 이동의 동작이 달라져야 했습니다. 하지만 각 오브젝트가 진행 여부를 따로 관리하면서, 저장·로드와 씬 재방문 과정에서 상태를 일관되게 유지하기 어려웠습니다.

SOLUTION

이벤트 완료 여부를 하나의 `GameFlag` 로 관리하는 `ScriptableObject` 기반 상태 시스템을 만들었습니다. `EnemyFindZone` 과 `DoorTrigger` 가 동일한 `GameFlag`를 참조하도록 하고, 저장된 게임을 불러올 때는 `RestoreFromSave()` 로 진행 상태를 복원하도록 연결했습니다.

```
GameFlag.cs - ScriptableObject 기반 상태

public class GameFlag : ScriptableObject
{
    private bool _value;
    public bool Value => _value;

    public void Set(bool value)
    {
        _value = value;
        GameFlags.Set(name, value);
    }
}
```

RESULT — 이벤트를 완료한 뒤 다시 방문하거나 게임을 저장·로드해도, 이미 진행한 이벤트의 상태가 유지되도록 만들었습니다.

02

플레이어를 제대로 스폰했는데, 왜 바닥으로 떨어질까?

시네마틱을 본 직후 다음 씬으로 이동하면 플레이어가 지정한 문 앞이 아니라 바닥에 떨어지는 버그가 발생했습니다. 재방문 시에는 정상 작동해 처음엔 `SpawnPoint` 문제로 오해했습니다.

CAUSE

`MonologueManager` 가 `DontDestroyOnLoad` 로 유지되면서 다음 씬으로 넘어간 뒤에도 `isControlBlocked = true` 상태가 남아있었습니다. 이동 로직은 멈췄지만 `Rigidbody2D`의 중력은 계속 적용돼, 스폰 직후 아래로 떨어지고 있었습니다.

SOLUTION

씬을 이동시키는 `MoveScene()` 에서 남아있던 조작 제한 상태를 명시적으로 해제했습니다.

```
MonologueManager.cs - MoveScene()

private void MoveScene()
{
    monologueManager.DialogueFinished -= MoveScene;
    ClearSky.Player.isControlBlocked = false;
    thePlayer.pendingEntrance = destinationEntrance;
    StartCoroutine(ChangeSceneWithDelay());
}
```

WHAT I LEARNED — 프로젝트 규모가 커질수록 씬 전환·저장/로드·대화·물리처럼 서로 다른 시스템이 만나는 지점에서 새로운 문제가 발생한다는 것을 경험했습니다. PH-1을 개발하며 기능 하나의 코드만 보는 것이 아니라, 게임 전체에서 상태가 어떤 순서로 변경되고 전달되는지를 추적하며 문제를 해결하는 방법을 배웠습니다.

키보드 대신, 실제 손으로 요리하는 게임

웹캠으로 플레이어의 손을 인식해 잡기·썰기·젓기·뒤집기 등
실제 손동작을 게임 조작으로 연결한 요리 게임입니다.

MediaPipe로 21개의 손 랜드마크를 실시간 추적하고, 손의 위치·
움직임·쥐는 동작을 분석해 손 아바타와 조리 도구를 직접
조작하도록 구현했습니다. HandLandmarker 를 LIVE_STREAM 모드로
실행하고, 각 랜드마크로 위치와 동작 상태를 계산합니다.

카메라에 손을 비추는 데서 끝나지 않고, 손을 쥐면 도구를 잡고 →
실제로 움직이면 → 그 움직임에 따라 조리 과정이 진행되는 것을
목표로 제작했습니다.



HAND TRACKING

MediaPipe 기반 21개 손 랜드마크 실시간 추적

GRIP

손가락이 접힌 정도와 손끝-손바닥 거리를 이용해 쥐기
동작 판별

MOTION

손의 이동 방향과 속도를 분석해 썰기·젓기·뒤집기 동작
으로 변환

DEPTH

손바닥의 화면상 크기로 카메라와 손 사이의 깊이값 추
정



TECH

- Python
- OpenCV
- MediaPipe
- NumPy
- Kalman Filter
- Pygame

CORE IMPLEMENTATION

카메라 속 손동작을 어떻게 '게임 조작'으로 바꿀까?

01

손을 인식하는 것만으로는 게임을 조작할 수 없었습니다

MediaPipe는 손의 랜드마크 좌표만 줄 뿐, 도구를 잡고 있는지·어느 속도로 움직이는지·무엇을 조작하는지는 판단해주지 않았습니다.

SOLUTION

21개 랜드마크를 가공해 게임에서 쓸 `HandState` 로 변환, CV 결과를 게임 입력 데이터로 추상화했습니다.

```
hand_tracker.py - HandState

@dataclass
class HandState:
    detected: bool = False
    x, y, z: float = 0.0 # world pos
    vx, vy: float = 0.0 # Kalman 속도
    gripped: bool = False
    screen_x, screen_y: int = 0 # 1280x720
    landmarks: list = field(default_factory=list)
```

03

보이는 손과 실제 판정 위치를 일치시켰습니다

화면에는 랜드마크 기반 손 아바타가 표시되는데, 판정 기준점과 아바타 위치가 다르면 "손은 칼 위에 있는데 왜 안 잡히지?" 같은 문제가 생깁니다.

SOLUTION

21개 랜드마크 전체의 중심점을 화면상 손 위치로 사용해, 이 좌표가 손 아바타의 중심과 정확히 일치하도록 만들었습니다.

```
hand_tracker.py - screen position

avg_nx = sum(lm.x for lm in lms) / len(lms)
avg_ny = sum(lm.y for lm in lms) / len(lms)
sx = int(avg_nx * 1280); sy = int(avg_ny * 720)
```

RESULT — 보이는 손의 위치 = 실제 상호작용 판정 위치가 되도록 입력과 UI를 맞췄습니다.

02 — 손의 움직임을 각 요리 동작에 맞게 해석했습니다

모든 제스처를 하나의 속도 임계값으로 처리하지 않고, 요리 동작마다 다른 방식으로 판정합니다.

CUT

방향 전환 감지

칼을 쥔 상태에서 손의 수직 속도(vy)가 임계값을 넘겨 위↔아래로 반전되는 순간을 한 번의 칼질로 판정합니다.

```
prev_vy > SPEED and vy < -SPEED
→ cuts += 1
```

STIR

각도 누적 판정

그릇 중심 기준 손의 각도를 매 프레임 계산해 변화량을 누적하고, 합이 2π 가 될 때마다 한 바퀴로 판정합니다.

```
angle = atan2(dy, dx)
total_ang += delta → stirs++
```

FLIP

상승 속도 감지

팬을 쥔 상태에서 손이 짧은 시간에 일정 속도 이상 위로 움직이면 뒤집기 동작으로 판정합니다.

```
hand.gripped and vy > FLIP_SPEED
→ flips += 1
```

MOTION STABILIZATION

x, y, vx, vy를 상태값으로 갖는 Kalman Filter로 손 움직임의 속도를 추정

GAME FLOW

MENU→TUTORIAL→COUNTDOWN→PLAYING→RESULT→RANKING 상태 관리

SCORE & RANKING

연속 성공 Combo multiplier, 조리별 점수, JSON 기반 최고 기록 랭킹 저장

WHAT I LEARNED — 손을 인식하면 곧바로 게임 조작에 쓸 수 있을 것이라 생각했지만, 실제로는 인식된 좌표를 그대로 쓰는 것보다 플레이어가 의도한 동작이 무엇인지 해석하는 과정이 더 중요했습니다. 손 랜드마크에서 위치·속도·꺾기 상태를 추출해 썰기·젓기·뒤집기라는 서로 다른 게임 규칙으로 변환하며, 컴퓨터 비전 기술 자체뿐 아니라 그 데이터를 플레이어가 직관적으로 이해할 수 있는 인터랙션으로 설계하는 과정을 경험했습니다.

OTHER GAMES

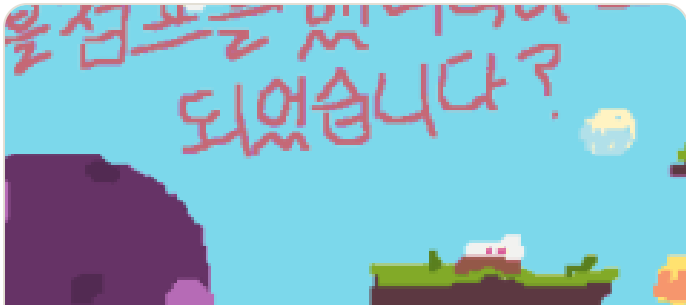
Crazy Commuter · PH-1 · Cooking Papa 외에도, 꾸준히 게임을 만들어왔습니다



2020 · FIRST GAME

포도씨의 모험 (3D)

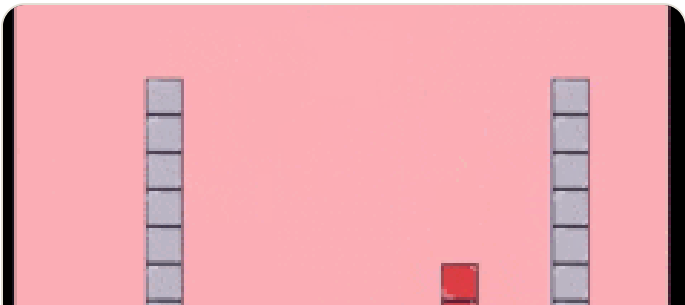
Unity · 3D 플랫폼



2020

포도씨의 모험 (2D)

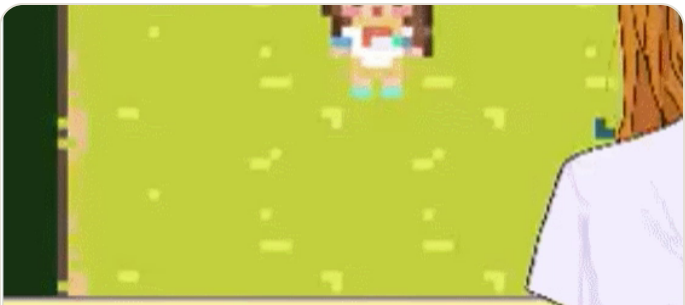
Unity · 2D 플랫폼



2020

테트리스

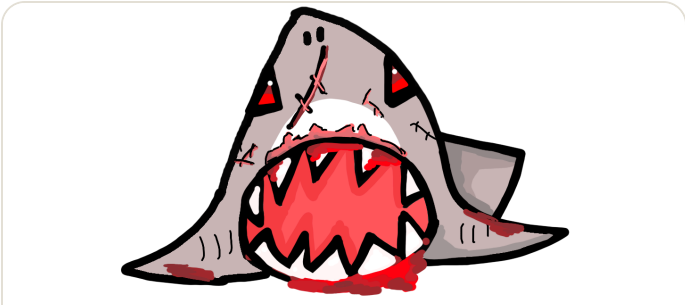
Unity · 퍼즐



2021

뽕머랭님 팬게임

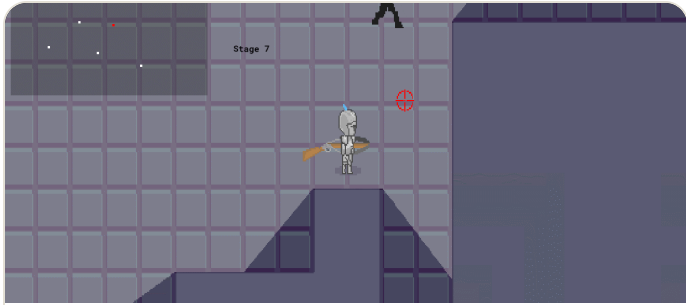
Unity · 탑다운 RPG



2021

Shark Timer Game

p5.js · 협업



2022

빛이있으라

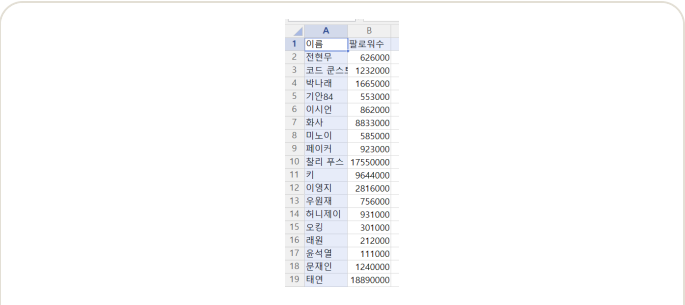
GameMaker Studio · 팀장



2023

Arrow Master

p5.js · 2인용 대결



2023

더 인기, 더 팔로워

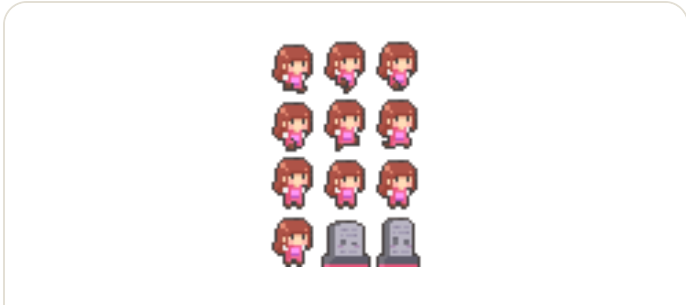
Python · tkinter



2024 · 외주

내 과일을 받아줘

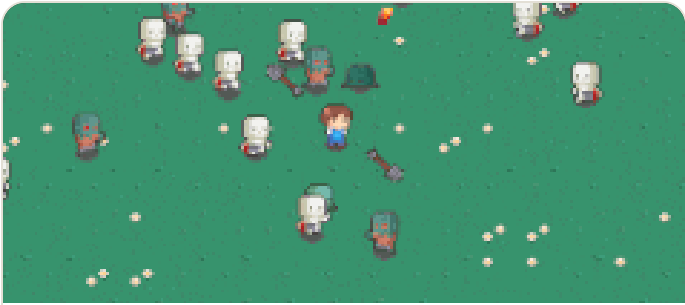
Processing · 캐주얼



2024

빼제로의 풍선

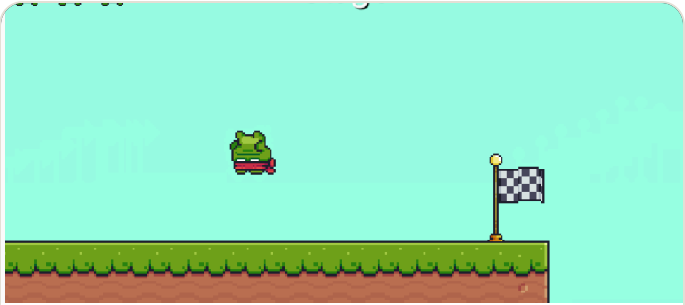
Unity · 2D 공포



2024

뱀서라이크

Unity · 로그라이크



2025

함정 피하기 게임

Unity · 2D 플랫폼

2020년부터 지금까지, 장르와 도구를 가리지 않고 꾸준히 게임을 만들어왔습니다.
더 많은 프로젝트와 플레이 영상은 아래 링크에서 확인하실 수 있습니다.

GAME PORTFOLIO
hyewang5617.github.io/Portfolio